4CS015 – Workshop #5 TO BE SUBMITTED AS PART OF YOUR PORTFOLIO – please note this workshop covers material from lectures 3,4,5 and 6.

Name: Sagar Mishra

Student ID: 2606063

This is a marked workshop. **It forms the second part of your portfolio**. You will need to complete the workshop and then submit a copy of this document with a title that follows the following format ("YOURNAME_Uni Id_wsp5.pdf"), via CANVAS, by the deadline.

Table of Contents

a. Use A= 11 B=4, complete the following table in binary *(8 marks)*: 6

b. Use the logical operations (bitwise). Manually prove each operation has returned the correct result (8 marks), show your working out for all off the above answers. 6

c. Build the circuit above *(10 marks):* 8

d. Complete the following timing diagram by filling in the table spaces with '1' or '0'. (16 marks) 8

e. Next, describe what the circuit does. (8 marks) 8

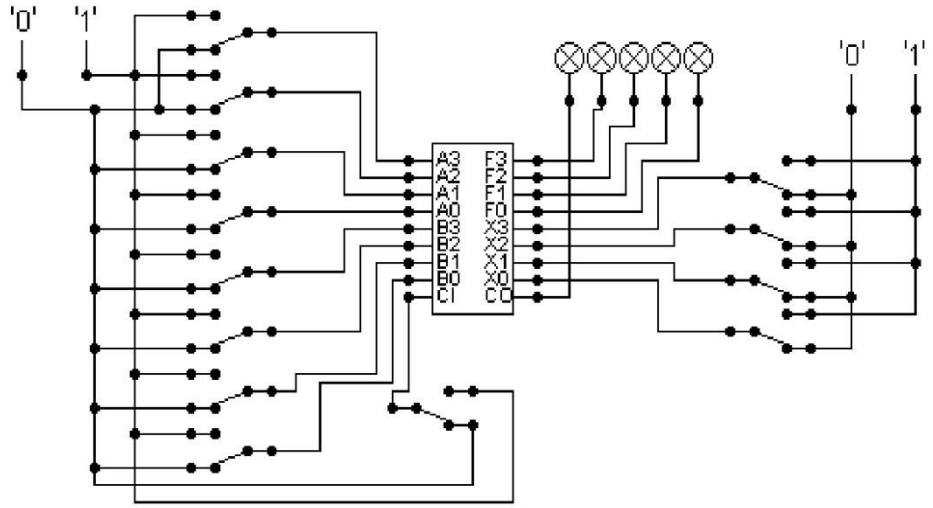
f. Describe what these circuits do, please provide details by clock pulse and the changing of the shift/load input (all the circuits do same thing, except the logsim version has no serial in). *(10 marks)* 10

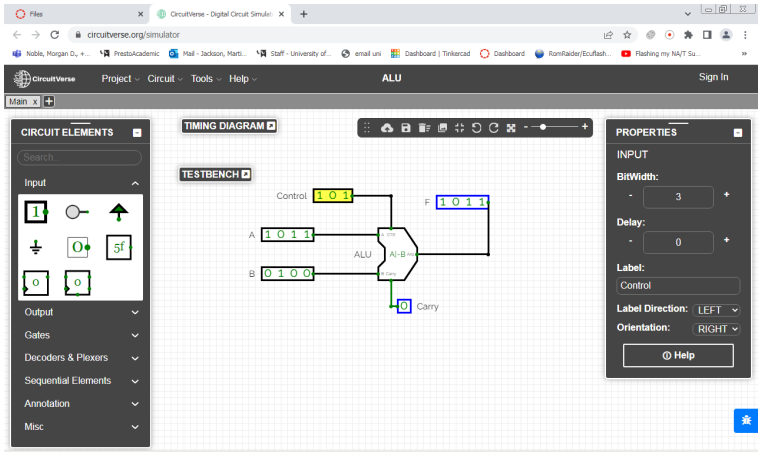
g. Next, design a circuit that takes 4 inputs that takes the Clock X and the Output Y and decodes Y into 4 output indicators so that they match D0 – D3. Insert the GIF output of your design in the space below. Highest marks will go to those who design the circuit such that it AUTOMATICALLY stops (not pauses) when the input to the circuit matches the output to the circuit. Screenshot your circuit and upload your logsim/circuitVerse file. Describe how your circuit works? *(40 marks)* 11

Workshop tasks:

Arithmetic Logic Unit

Load the Logisim Arithmetic Logic Unit Circuit **alu.cct** from inside the logsim application (You'll find it in the logsim folder) or the ALU for CircuitVerse. ALU.cv. It should look like these:





The circuit behaves like a simple arithmetic logic unit. The inputs A0-A3 represent a 4-bit binary number. Inputs B0-B3 represent another binary number. A0 and B0 are the least significant bits respectively. The following table details the functions supported by the chip.

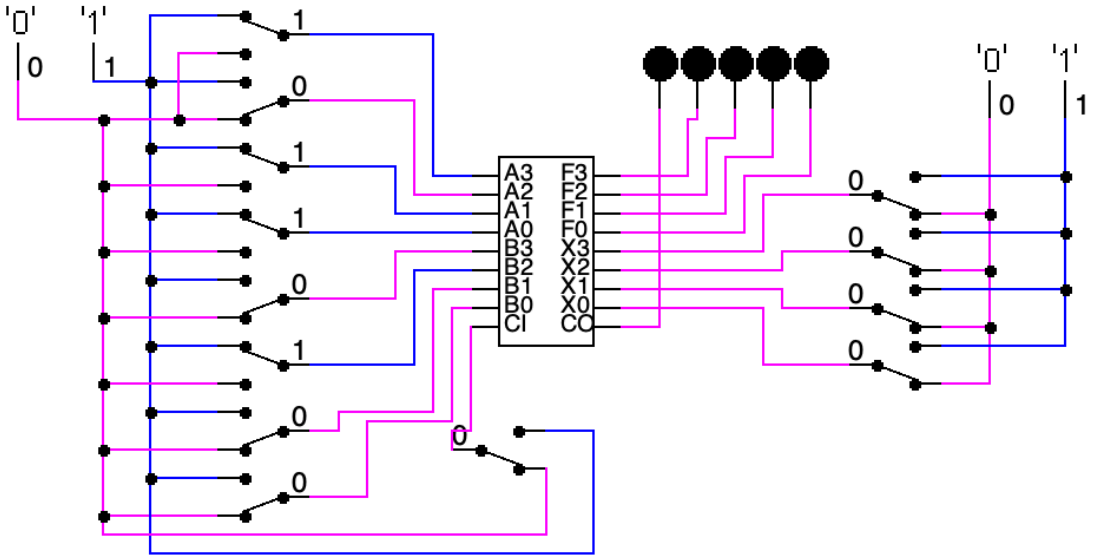
Logsim control settings

Function	AND	OR	XOR	NAND	NOR	NOT A	ADD	SUBTRACT
X3 – X0	0000	0001	0010	0011	0100	0101	1010	1011

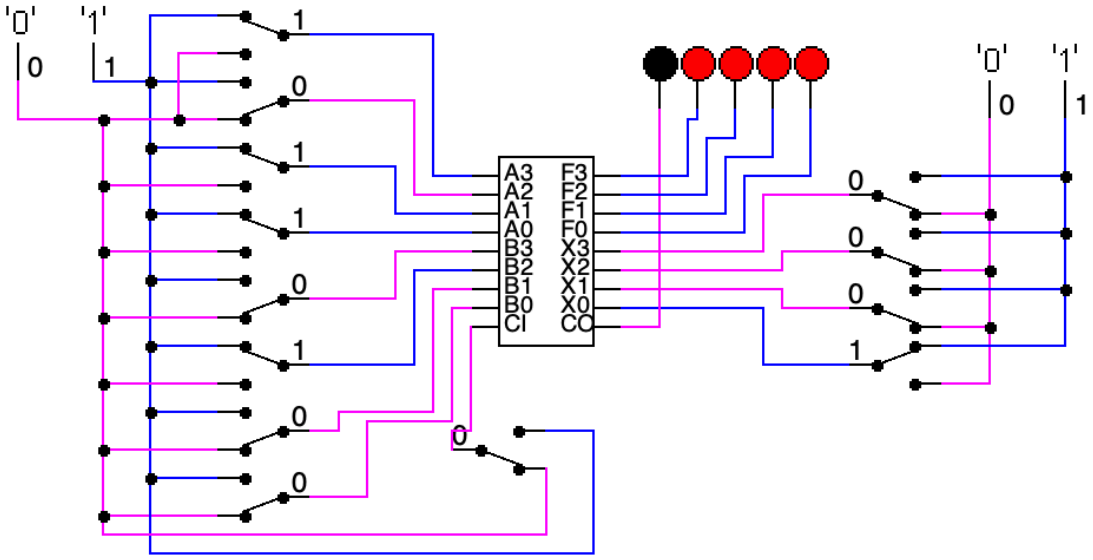
CircuitVerse control settings

Opcode	**Operation**
0 0 0	A & B AND
0 0 1	A | B OR
0 1 0	A + B ADD
0 1 1	No spec
1 0 0	A & ~B A AND notB
1 0 1	A | ~B A OR notB
1 1 0	A – B SUBTRACT
1 1 1	SLT A<B **(Set Less Than): If A is less than B, output is 1 else output is 0.**

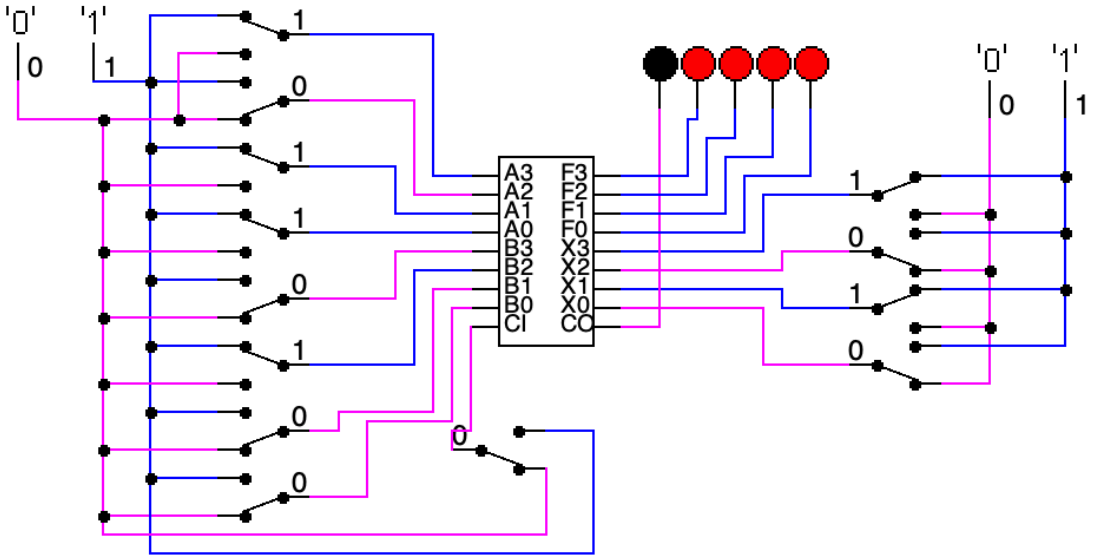
1. AND operator



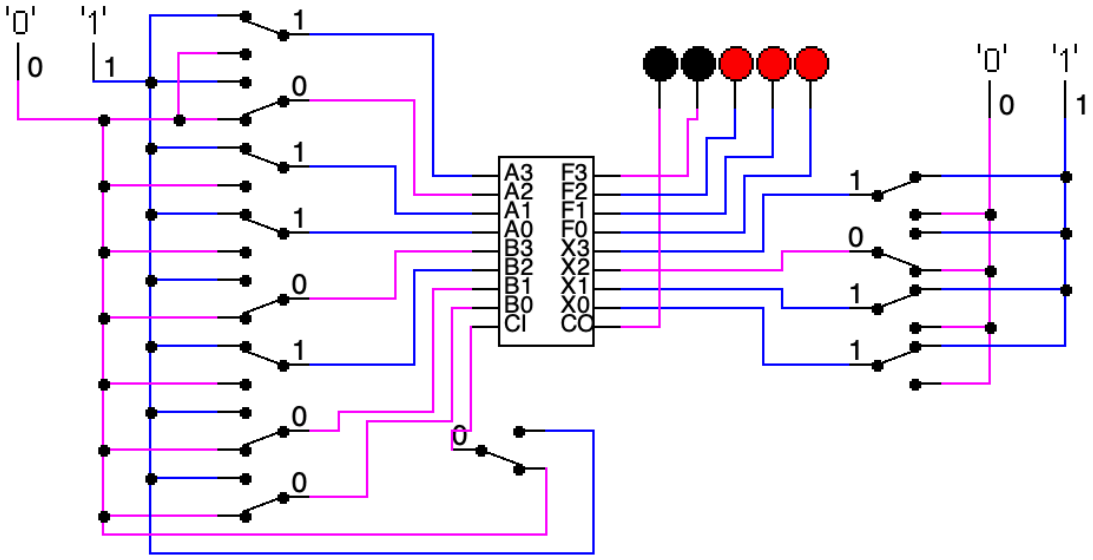
2. OR operator



3. ADD



4. SUBTRACT



a. Use A= 11 B=4, complete the following table in binary *(8 marks)*:

FUNCTION	OUTPUT
AND	0000
OR	1111
ADD	1111
SUBTRACT	0111

b. Use the logical operations (bitwise). Manually prove each operation has returned the correct result (8 marks), show your working out for all off the above answers.

Example:

1 0 0 0
<u>0 1 0 0</u> AND OPERATION
0 0 0 0 RESULT

→ **<u>AND:</u>**

Given A = 11, B = 4
Here, A = 11 = $(1011)_2$
B = 4 = $(0100)_2$

1 0 0 0
<u>0 1 0 0</u> AND OPERATION
0 0 0 0 RESULT

<u>OUTPUT</u>

A AND B

1 AND 0 → 0
0 AND 1 → 0
1 AND 0 → 0
1 AND 0 → 0

→ **<u>OR:</u>**

Given A = 11, B = 4 F
Here, A = 11 = $(1011)_2$
B = 4 = $(0100)_2$

A OR B
1 0 0 0
<u>0 1 0 0</u> OR OPERATION
1 1 1 1 RESULT

<u>OUTPUT</u>

A OR B

1 OR 0 → 1
0 OR 1 → 1
1 OR 0 → 1
1 OR 0 → 1

→ **ADD:**

Given A = 11, B = 4
Here, A = 11 = $(1011)_2$
B = 4 = $(0100)_2$

Here, 11+ 4 = 15
In Binary, $(1011)_2 + (0100)_2 = (1111)_2$

1 0 0 0
0 1 0 0 ADD OPERATION
1 1 1 1 RESULT

Let's check,
Is $(15)_{10} = (1111)_2$?

Converting $(1111)_2$ into decimal number system:
$= 1 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0$
$= (15)_{10}$

OUTPUT

A	ADD	B		
1	ADD	0	→	1
0	ADD	1	→	1
1	ADD	0	→	1
1	ADD	0	→	1

→ **SUBTRACT:**

Given A = 11, B = 4
Here, A = 11 = $(1011)_2$
B = 4 = $(0100)_2$

Here, 11 - 4 = 7
In Binary, $(1011)_2 - (0100)_2 = (0111)_2$

1 0 0 0
0 1 0 0 SUBTRACT OPERATION
0 1 0 0 RESULT

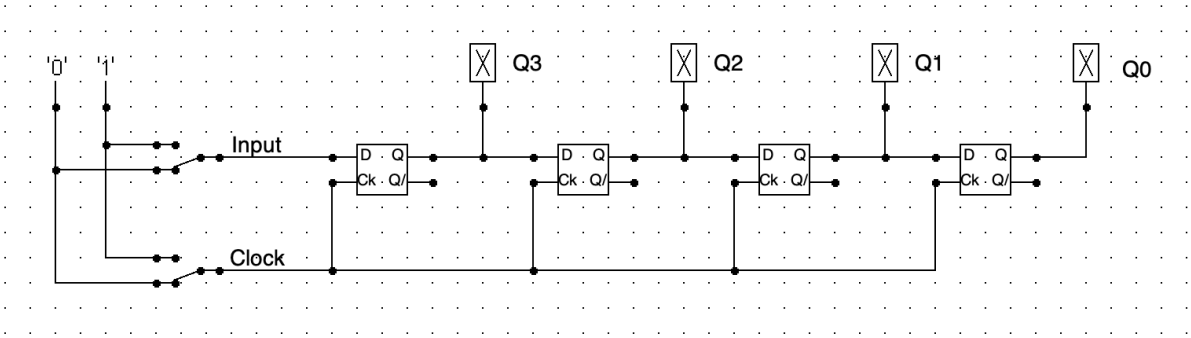
Let's check,
Is $(7)_{10} = (0111)_2$?

Converting $(1111)_2$ into decimal number system:
$= 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 1 * 2^0$
$= (7)_{10}$

OUTPUT

A	SUBTRACT	B		
1	SUBTRACT	0	→	0
0	SUBTRACT	1	→	1
1	SUBTRACT	0	→	1
1	SUBTRACT	0	→	1

c. Build the circuit above *(10 marks):*



d. Complete the following timing diagram by filling in the table spaces with '1' or '0'. (16 marks)

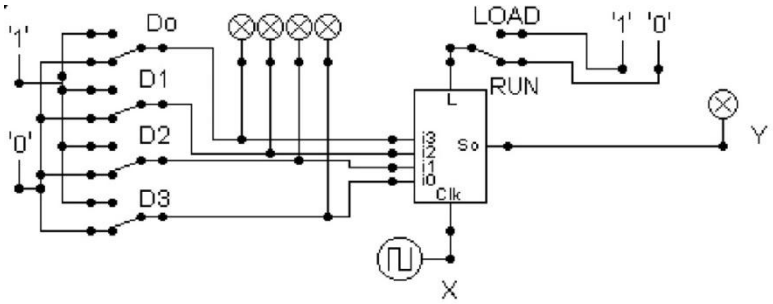
input	1	1	0	0	1	1	1	1	0
Clock	0	1	0	1	0	1	0	1	0
Q3	1	1	1	1	1	1	1	1	1
Q2	1	1	1	1	1	1	1	0	0
Q1	1	1	1	1	1	0	0	1	1
Q0	1	1	1	0	0	1	1	1	1

e. Next, describe what the circuit does. (8 marks)

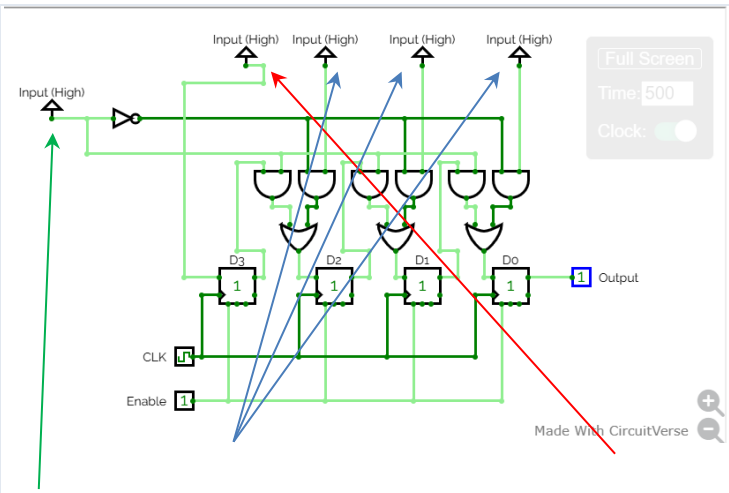
→ The above circuit is 4-bit serial in parallel-out (SIPO) shift register, built using D-type flipflop with a common clock. The above circuit shows 4-bit output (Q3, Q2, Q1, Q0) based on input and a common clock. Also, the above table shows the timing diagram of the circuit. Each flipflop stores one bit of data, and all flipflops share a common clock signal, ensuring that they operate synchronously. In simple terms, this circuit takes data one bit at a time and moves it through the flipflops whenever the clock changes. The input is connected to the first D flipflop, which produces output Q3. All the flipflops use the same clock and therefore they work together. When the clock changes from 0 to 1, each flipflop saves the value present at its input. The new input value goes into Q3, the previous value of Q3 moves to Q2, the value of Q2 moves to Q1, and the value of Q1 moves to Q0. In this way, the data shifts one step forward each time the clock is pressed. After four clock pulses, the circuit holds 4-bits of data at outputs Q3, Q2, Q1, and Q0. These outputs can be read at the same time, which is why it is called a serial in parallel-out register**.** The preset and clear inputs are turned off, so the circuit only changes when the clock and input are used. This type of circuit is commonly used for temporary data storage**,** data transfer and converting serial data into parallel data in digital systems.

Combining everything we have covered so far (lectures 3-6).

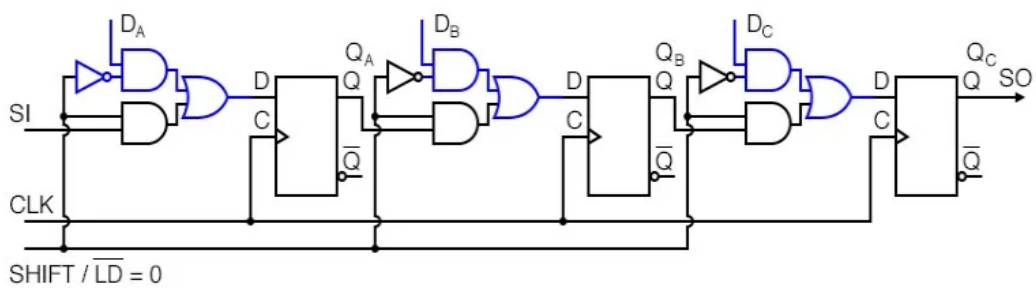
Look at the circuits below:



Logsim



Circuitverse - Dflipflops used with load circuit 3-bit input (first input is serial-in not used). The circuit has another input that allows the selection between **load input** and **shift register** mode. (ref: circuitverse diagram).



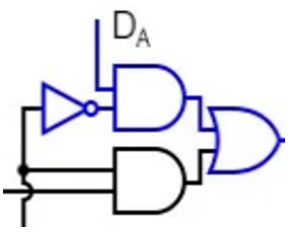
Above is a 3-stage shift register. A stage consists of a type **D** Flip-Flop for storage, and an AND-OR selector to determine whether data will load in parallel, or shift stored data to the right. These elements will be replicated for the number of stages required. We show here only three stages. SI has been provided to allow us to cascade shift registers.

f. Describe what these circuits do, please provide details by clock pulse and the changing of the shift/load input (all the circuits do same thing, except the logsim version has no serial in). *(10 marks)*

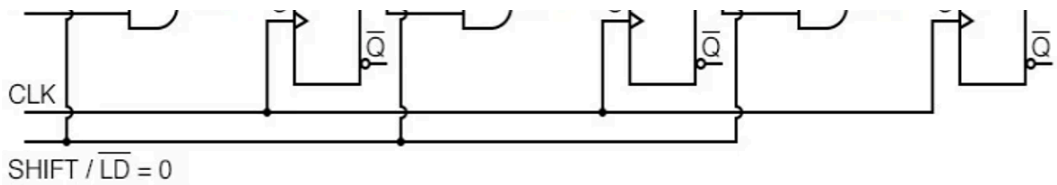
→ The circuit shown above is a 3-stage shift register that can either load a parallel input or shift its content side by side right by one bit on each clock pulse. Each stage consists of a D-flipflop for storage and an AND-OR selector to determine whether data will load in parallel or shift stored data to the right. There are three storage stages Q_A, Q_B, and Q_C connected with flipflops respectively. Shift/Load selects between two modes on every clock pulse.

- **Load mode:** When Shift/Load = 1, then the flipflops takes the three parallel inputs D_A, D_B, and D_C simultaneously.
- **Shift mode:** When Shift/Load = 0, then the flipflop shifts the data to the right, Q_A takes serial-in, Q_B takes Q_A, and Q_C takes Q_B.

Each stage has AND-OR selector in front of every D input respectively. One AND gate passes the corresponding load input of D_x in Load mode, and the other AND gate passes the previous stage's Q (or SI for Q_A) in Shift mode. The OR gate combines these two inputs coming from two AND gates output and passes it to respective flipflop according to which the flipflop either loads data or shifts data forward.



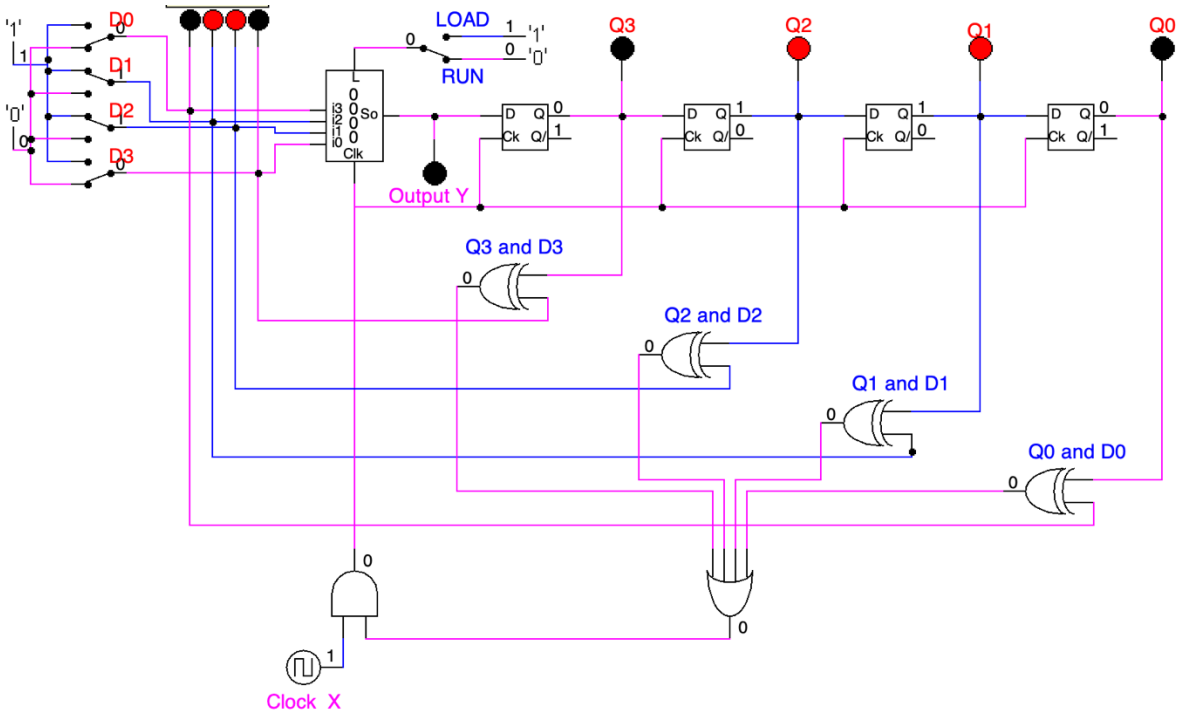
In this circuit the clock controls when will the circuit shifts or loads data and nothing changes until the instant when clock signal changes level. On each clock edge, the flipflop takes its D input and updates it to Q output at the same instant. In Load mode, then edge makes register copy the 3 D input bits and paste it into Q respectively. On Shift mode, the edge makes ach flipflop take the bit from its left Q or SI, so all stored bits shift one place right from where it is.



The main purpose of this circuit is to store a 3 but value at once and shift those bits one step right on each clock pulse so they can be used or sent out in a different form. This circuit also lets system change data from parallel form to serial form. Hence, the circuit above is a 3-bit shift register with parallel load whose main purpose is to store 3-bit input at a time and shift it serially on each clock pulse.

g. Next, design a circuit that takes 4 inputs that takes the Clock X and the Output Y and decodes Y into 4 output indicators so that they match D0 – D3. Insert the GIF output of your design in the space below. Highest marks will go to those who design the circuit such that it <u>AUTOMATICALLY</u> stops (not pauses) when the input to the circuit matches the output to the circuit. Screenshot your circuit and upload your logsim/circuitVerse file. Describe how your circuit works? *(40 marks)*

→ Below is the circuit that takes 4 inputs that takes Clock X and the Output Y and decodes it into 4 output indicators so that they match D0-D3, and stops automatically when the input to the circuit matches the output to the circuit:



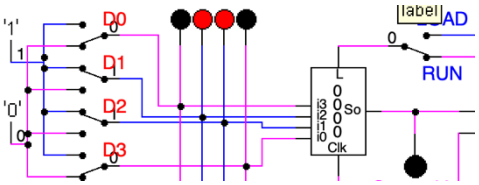
<u>Component used in the above circuit:</u>

- Parallel 4 inputs.
- Shift Register.
- 4 X-OR Gates.
- OR gate with 4 inputs.
- AND gate.
- Clock.
- 9 LED
- 4 D-flipflops

Description:

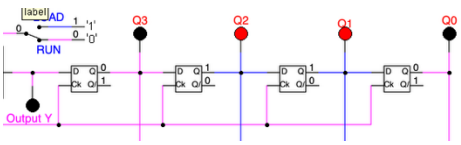
In the above circuit, the mentioned components are connected in such a way that it takes 4 input D0-D3 and Clock X in a shift register and generates output Y and then uses 4 D-flipflops to decode it into 4 serial output Q0-Q3. The 4 X-OR gates, OR gate and an AND gate is used to make the circuit stop automatically when the input to the circuit matches the output to the circuit. Above image shows when D1 and D2 are red and load is once given then the output Y is obtained and decoded into 4 serial outputs where respectively Q1 and Q2 becomes red and stops automatically as the input to the circuit matches the output to the circuit.

Input Section:



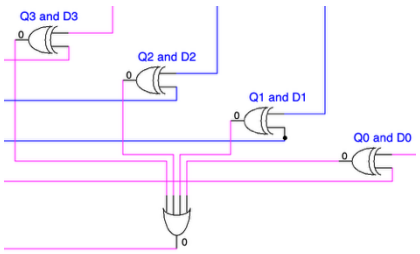
The input section consists of 4-bit parallel data input D0-D3, which represents the target value that we want to compare the circuit with. The switches are connected to each bit input D, to manually set the binary values (0 or 1) for D0-D3. Load is used to load the data input to the shift register, when Load = 1, the input data is loaded into the shift register and used to initialize the circuit. Run is used to start the circuit to operate, when Run = 1, the circuit starts operating and clock is allowed, when Run = 0 the circuit stops manually

Storage section:



The Series of D-flipflops that decodes output into 4 serial output is called as decoder. These are the storage sections(register) that Loads the current output value of the circuit and the value updates on every clock pulse. The flipflops used in this 4-bit register, each store one bit of data. Q3-Q0 are the 4-bit output as shown in the circuit. The LED of Output Y shows the current output from shift register and then the output is stored using decode.

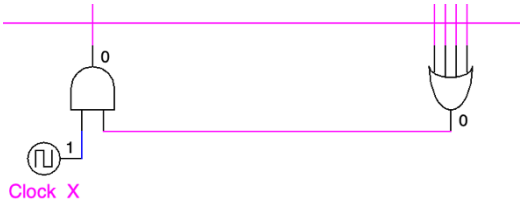
Comparator section:



The 4 X-OR gate's output is connected to the OR gate that takes 4 input and sends an output signal to the AND gate where the clock impulse is decided to go forward or stop there. If the OR gate sends 0 the AND will not let clock impulse value to go forward as AND gate only sends signal when both inputs are 1. The 4 X-OR gates generate signal until the input value matches the output value. When inputs are different output is one, and when inputs are same output is 0. Each X-OR gate compares the one bit of Q with one bit of D, and when all bits match, all X-OR gate outputs become 0. XOR compares,

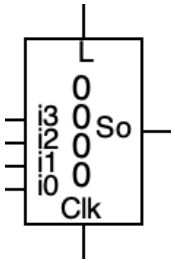
$Q0 \oplus D0$ $\quad$ $Q1 \oplus D1$ $\quad$ $Q2 \oplus D2$ $\quad$ $Q3 \oplus D3$.

When all the outputs of X-OR gates are 0 the OR gate send signal 0 to the AND gate. OR gate checks the mismatch and it is also called as mismatch detector in this circuit. It sends signal 1 until the input matches output to the circuit. The AND gate combines Clock signal and Comparator output and sends signal to shift register. The AND gate sends signal until the input to the circuit matches the output to the circuit.



Shift Register:

A shift Register is a sequential digital circuit which is made from group of flipflops connected in a series that stores and binary data one bit at a time on each clock pulse. In this circuit a 4-bit shift register loads data in parallel and shifts it during RUN mode and the comparator automatically stops this when the input to the circuit matches the output to the circuit.



Why the circuit automatically stops when input matches output?

- Each X-OR gate compares 1 bit:
  If Q = D, XOR output = 0

- When all bits match, All XOR gate output becomes 0.

- The 4 inputs of OR gate becomes 0, then output of OR gate becomes 0.
  Output = XOR OR XOR OR XOR OR XOR
  = 0 OR 0 OR 0 OR 0
  = 0

- Now, AND gate has two inputs, one from clock pulse and another from OR gate i.e. 0.
  Clock (1/0) AND 0 = 0

- Since no Clock reaches the flipflops, the register remains constant and stops changing. Hence output remains fixed.

Thus, the circuit automatically stops when the input to the circuit matches the output to the circuit.